

Leetcode 150 - Arrays/String

1. Java

Arrays In Java, an array is a container object that holds a fixed number of values of a single type.

- Key Traits: Once created, its length cannot change. It is strictly typed (e.g., an `int[]` can only hold integers).
- Syntax: `int[] numbers = {1, 2, 3};`

Strings A String in Java is an object that represents a sequence of characters.

- Key Traits: They are immutable, meaning once a String object is created, its value cannot be changed. Modification creates a new object in memory.
- Syntax: `String greeting = "Hello";`

2. Python

Arrays (Lists) Python does not have native "arrays" in the same strict sense as Java. Instead, it uses Lists.

- Key Traits: Lists are dynamic (can grow/shrink) and heterogeneous (can hold mixed data types like integers and strings together).
 - Note: Python has a specific array module for C-style fixed-type arrays, but it is rarely used compared to Lists or NumPy arrays.
- Syntax: `numbers = [1, 2, "three"]`

Strings A String in Python is a sequence of Unicode characters.

- Key Traits: They are immutable. Python has no distinct char data type; a single character is simply a string of length 1.
- Syntax: `greeting = 'Hello'` or `greeting = "Hello"`

88 - Merge Sorted Array

This problem, "Merge Sorted Array," is a classic array manipulation task often encountered in technical interviews. It tests your understanding of sorted arrays and in-place modifications.

Problem Understanding

You are given two integer arrays, `nums1` and `nums2`, both sorted in **non-decreasing order**.

- `m` represents the number of elements in `nums1` that are actually part of the sorted sequence.
- `n` represents the number of elements in `nums2`.

The goal is to **merge** `nums2` into `nums1` such that `nums1` contains all elements from both arrays, also in non-decreasing order. A key constraint is that the merge must happen **in-place** within `nums1`.

To accommodate this, `nums1` is pre-allocated with a length of `m + n`. The first `m` elements are the valid sorted numbers from the original `nums1`, and the last `n` elements are typically `0`s (or any placeholder value) that should be ignored, as they will be overwritten by elements from `nums2` during the merge. The function itself does not return anything (`void` in Java, `None` in Python) because it modifies `nums1` directly.

Example: `nums1 = [1, 2, 3, 0, 0, 0]`, `m = 3` `nums2 = [2, 5, 6]`, `n = 3`

Output: `[1, 2, 2, 3, 5, 6]`

Brute-Force Approach

A straightforward way to solve this problem is to first collect all elements into `nums1` and then sort `nums1`.

1. **Copy Elements:** Take all elements from `nums2` and place them into the `n` empty slots at the end of `nums1`.
 - Specifically, copy `nums2[i]` to `nums1[m + i]` for `i` from `0` to `n-1`.
2. **Sort:** Sort the entire `nums1` array (which now contains all `m+n` elements).

Example Walkthrough: `nums1 = [1, 2, 3, 0, 0, 0]`, `m = 3` `nums2 = [2, 5, 6]`,
`n = 3`

1. Copy `nums2` into `nums1`:

- `nums1[3] = nums2[0]` (i.e., 0 becomes 2)
- `nums1[4] = nums2[1]` (i.e., 0 becomes 5)
- `nums1[5] = nums2[2]` (i.e., 0 becomes 6) `nums1` becomes `[1, 2, 3, 2, 5, 6]`

2. Sort `nums1`:

- `nums1 [1, 2, 3, 2, 5, 6]` sorted becomes `[1, 2, 2, 3, 5, 6]`

Complexity Analysis

- **Time Complexity:** $O((m+n) \log(m+n))$
 - Copying `n` elements takes $O(n)$ time.
 - Sorting an array of `m+n` elements generally takes $O((m+n) \log(m+n))$ time using efficient comparison-based sorting algorithms (like quicksort or merge sort).
 - Overall, the sorting step dominates, so the complexity is $O((m+n) \log(m+n))$.
- **Space Complexity:** $O(1)$ (or $O(m+n)$ in worst-case for some sort implementations)
 - We are modifying `nums1` in-place. We don't use any auxiliary data structures whose size scales with `m` or `n` for copying.
 - However, the `sort` function itself might use auxiliary space. For example, Python's `list.sort()` (Timsort) can use $O(N)$ space in the worst case, and Java's `Arrays.sort()` uses $O(\log N)$ stack space for primitives (dual-pivot quicksort) and $O(N)$ for objects (merge sort). When discussing competitive programming problems, " $O(1)$ space" usually refers to *auxiliary* space, ignoring the space taken by the input itself and the call stack of the system's sorting algorithm unless explicitly noted. Given this, it's often considered $O(1)$ auxiliary space.

Brute-Force Code Solutions

While not optimal, here's how you'd implement the brute-force approach for completeness:

Java:

```
import java.util.Arrays; // Needed for Arrays.sort()

class Solution {
    public void merge(int[] nums1, int m, int[] nums2, int n) {
        // Step 1: Copy elements from nums2 into the end of nums1
        for (int i = 0; i < n; i++) {
```

```

        nums1[m + i] = nums2[i];
    }

    // Step 2: Sort the entire nums1 array
    Arrays.sort(nums1);
}
}

```

Python:

```
from typing import List
```

```
class Solution:
```

```
    def merge(self, nums1: List[int], m: int, nums2: List[int], n: int) -> None:
```

```
        """
```

```
        Do not return anything, modify nums1 in-place instead.
```

```
        """
```

```
        # Step 1: Copy elements from nums2 into the end of nums1
```

```
        # Pythonic way to do this is slicing
```

```
        nums1[m:] = nums2[:n] # Copies all elements of nums2 into nums1 starting at index m
```

```
        # Step 2: Sort the entire nums1 list
```

```
        nums1.sort()
```

Optimal Approach: Two Pointers (Merge from Back)

The optimal approach leverages the fact that both arrays are already sorted. Since `nums1` has enough space at its *end* to accommodate `nums2`, we can perform a merge operation by starting from the end of both arrays and placing the larger elements at the end of `nums1`. This strategy prevents overwriting elements in `nums1` that haven't been processed yet.

Strategy

1. Initialize Pointers:

- `p1`: Points to the last *actual* element of `nums1` (index `m - 1`).
- `p2`: Points to the last element of `nums2` (index `n - 1`).
- `write_idx`: Points to the last available position in `nums1` where an element can be written (index `m + n - 1`). This is where the largest element will go.

2. **Merge Loop:** Iterate while both `p1` (for `nums1`) and `p2` (for `nums2`) are non-negative:

- Compare `nums1[p1]` and `nums2[p2]`.
- If `nums1[p1]` is greater than or equal to `nums2[p2]`:
 - Place `nums1[p1]` at `nums1[write_idx]`.
 - Decrement `p1` (move to the next smaller element in `nums1`).
- Else (if `nums2[p2]` is greater):
 - Place `nums2[p2]` at `nums1[write_idx]`.
 - Decrement `p2` (move to the next smaller element in `nums2`).
- In both cases, decrement `write_idx` (move to the next available position in `nums1` to the left).

3. **Handle Remaining Elements:** After the loop, one of the arrays might have elements left.

- If there are remaining elements in `nums2` (i.e., `p2 >= 0`): These elements are guaranteed to be smaller than any elements already placed in `nums1` (and thus smaller than any remaining elements in the initial part of `nums1`). Copy them directly to the beginning of `nums1` from `write_idx` downwards.
- If there are remaining elements in `nums1` (i.e., `p1 >= 0`): These elements are already in their correct sorted positions at the beginning of `nums1`, so no action is needed.

Example Walkthrough

`nums1 = [1, 2, 3, 0, 0, 0], m = 3` `nums2 = [2, 5, 6], n = 3`

Initial Pointers: `p1 = 2` (points to 3 in `nums1`) `p2 = 2` (points to 6 in `nums2`) `write_idx = 5` (points to the last 0 in `nums1`)

Merge Loop:

1. `nums1[2]` (3) vs `nums2[2]` (6). 6 is greater. `nums1[5] = 6` `nums1` is now `[1, 2, 3, 0, 0, **6**]` `p2` becomes 1 (points to 5) `write_idx` becomes 4
2. `nums1[2]` (3) vs `nums2[1]` (5). 5 is greater. `nums1[4] = 5` `nums1` is now `[1, 2, 3, 0, **5**, 6]` `p2` becomes 0 (points to 2) `write_idx` becomes 3

3. `nums1[2]` (3) vs `nums2[0]` (2). 3 is greater or equal. `nums1[3] = 3` `nums1` is now `[1, 2, **3**, 3, 5, 6]` `p1` becomes 1 (points to 2) `write_idx` becomes 2
4. `nums1[1]` (2) vs `nums2[0]` (2). 2 (from `nums1`) is greater or equal. `nums1[2] = 2` `nums1` is now `[1, **2**, 2, 3, 5, 6]` `p1` becomes 0 (points to 1) `write_idx` becomes 1
5. `nums1[0]` (1) vs `nums2[0]` (2). 2 is greater. `nums1[1] = 2` `nums1` is now `[1, **2**, 2, 3, 5, 6]` `p2` becomes -1 `write_idx` becomes 0

End of Merge Loop: `p2` is now -1, so the loop terminates.

Handle Remaining Elements: `p2` is -1, so `nums2` has no remaining elements. The `while (p2 >= 0)` condition is false. The elements from `nums1` (`p1` is 0, pointing to 1) are already in their correct places.

Final `nums1`: `[1, 2, 2, 3, 5, 6]`. This matches the example output.

Complexity Analysis

- **Time Complexity:** `O(m + n)`
 - We iterate through `nums1` (`m` elements) and `nums2` (`n` elements) effectively once, comparing and placing each element. Each pointer (`p1`, `p2`, `write_idx`) moves a total of `m+n` times combined.
- **Space Complexity:** `O(1)`
 - We are modifying `nums1` in-place and only using a few extra variables (pointers `p1`, `p2`, `write_idx`), whose number does not depend on the input size. This makes it an optimal space solution.

Optimal Code Solutions

Java:

```
class Solution {
    public void merge(int[] nums1, int m, int[] nums2, int n) {
        // Initialize pointers for nums1 (actual elements), nums2, and the write position in nums1
        int p1 = m - 1;      // Pointer for the last element in the actual part of nums1
        int p2 = n - 1;      // Pointer for the last element in nums2
        int write_idx = m + n - 1; // Pointer for the last available position in nums1
    }
}
```

```

// While there are elements to compare in both arrays
while (p1 >= 0 && p2 >= 0) {
    if (nums1[p1] >= nums2[p2]) {
        nums1[write_idx] = nums1[p1];
        p1--; // Move to the previous element in nums1
    } else {
        nums1[write_idx] = nums2[p2];
        p2--; // Move to the previous element in nums2
    }
    write_idx--; // Move to the previous write position in nums1
}

// If there are remaining elements in nums2, copy them to nums1
// (Elements remaining in nums1 are already in their correct places)
while (p2 >= 0) {
    nums1[write_idx] = nums2[p2];
    p2--;
    write_idx--;
}
}
}

```

Python:

```
from typing import List
```

```
class Solution:
```

```
    def merge(self, nums1: List[int], m: int, nums2: List[int], n: int) -> None:
```

```
        """
```

```
        Do not return anything, modify nums1 in-place instead.
```

```
        """
```

```
        # Initialize pointers for nums1 (actual elements), nums2, and the write position in nums1
```

```
        p1 = m - 1      # Pointer for the last element in the actual part of nums1
```

```
        p2 = n - 1      # Pointer for the last element in nums2
```

```
        write_idx = m + n - 1 # Pointer for the last available position in nums1
```

```
        # While there are elements to compare in both lists
```

```
        while p1 >= 0 and p2 >= 0:
```

```
            if nums1[p1] >= nums2[p2]:
```

```

    nums1[write_idx] = nums1[p1]
    p1 -= 1 # Move to the previous element in nums1
else:
    nums1[write_idx] = nums2[p2]
    p2 -= 1 # Move to the previous element in nums2
write_idx -= 1 # Move to the previous write position in nums1

# If there are remaining elements in nums2, copy them to nums1
# (Elements remaining in nums1 are already in their correct places)
while p2 >= 0:
    nums1[write_idx] = nums2[p2]
    p2 -= 1
    write_idx -= 1

```

27 - Remove Element

This problem asks us to remove all occurrences of a specific integer `val` from an integer array `nums` **in-place**. This means we should **modify the original `nums` array directly without allocating a new one**. The **order of the remaining elements can be changed**, and **elements beyond the new length `k` are not important**. Finally, we need to **return the new length `k`**.

Problem Specifics

The problem specifically states:

- **in-place**: Modify the `nums` array/list directly.
- **Order may change**: The relative order of the elements that are *not* `val` does not need to be preserved. This gives us flexibility.
- **Return `k`**: The function should return an integer `k`, representing the count of elements that are not equal to `val`. The elements beyond `k` in the `nums` array/list don't matter.

2. Brute-Force Approach

A straightforward, but not optimal, way to solve this problem is to iterate through the array. When we find an element equal to `val`, we try to remove it by shifting all subsequent elements one position to the left.

Algorithm

1. Initialize a counter `k` to 0 (which will eventually be the count of non-`val` elements).
2. Iterate through the array from the beginning to the end.

3. If `nums[i]` is equal to `val`:
 - Shift all elements from `nums[i+1]` to the end of the array one position to the left (i.e., `nums[j] = nums[j+1]` for `j` from `i` to `length - 2`).
 - Decrement the effective length of the array by 1.
 - Decrement the current index `i` by 1 because the element that moved into `nums[i]` position now needs to be checked.
4. If `nums[i]` is not equal to `val`:
 - Increment `k`.
5. Return `k`.

Time Complexity

- **$O(N^2)$** : In the worst-case scenario, every element might be `val`. For example, `nums = [3,3,3,3,3]`, `val = 3`.
 - When we find `val` at `nums[i]`, shifting the remaining `N-i-1` elements takes $O(N-i)$ time.
 - If `val` is found frequently (e.g., $N/2$ times) or if `val` is at the beginning of the array repeatedly, each shift operation could take $O(N)$ time.
 - Doing this `N` times results in an overall $O(N^2)$ time complexity.

Space Complexity

- **$O(1)$** : This approach modifies the array in-place, so it uses constant extra space.

3. Optimal Code Solutions (Two-Pointer Approach)

The most optimal approach uses a two-pointer technique. Since the order of elements may be changed and elements beyond the new length `k` don't matter, we can simply move all elements that are not `val` to the beginning of the array.

Algorithm Explanation

We'll use two pointers:

1. **`k` (Write Pointer / Slow Pointer)**: This pointer keeps track of the next position where a non-`val` element should be placed. It effectively marks the end of the "new" modified array. It starts at `0`.

2. **i (Read Pointer / Fast Pointer):** This pointer iterates through the entire original array, checking each element. It starts at 0.

Here's how it works:

- Initialize `k = 0`. This `k` will also serve as our count of elements not equal to `val`.
- Iterate `i` from 0 to `nums.length - 1` (or `len(nums) - 1` in Python).
- Inside the loop, if `nums[i]` is **not equal** to `val`:
 - It means we've found an element we want to keep.
 - We place this element at `nums[k]` (i.e., `nums[k] = nums[i]`).
 - Then, we increment `k` to prepare for the next non-`val` element.
- If `nums[i]` is **equal** to `val`:
 - We simply skip this element. We don't copy it, and we don't increment `k`. This effectively "removes" it from the logical array.
- After the loop finishes, `k` will hold the total number of elements that were not equal to `val`, and these elements will be correctly positioned at the beginning of the `nums` array (`nums[0]` to `nums[k-1]`).

Example Walkthrough

Let `nums = [3, 2, 2, 3]` and `val = 3`.

1. Initialize `k = 0`.
2. **i = 0:** `nums[0] = 3`. Is `3 != val` (which is 3)? No, `3 == 3`. Skip.
3. **i = 1:** `nums[1] = 2`. Is `2 != val`? Yes, `2 != 3`.
 - `nums[k] = nums[i] => nums[0] = nums[1] => nums[0] = 2`.
 - Increment `k => k = 1`.
 - `nums` is now `[2, 2, 2, 3]`.
4. **i = 2:** `nums[2] = 2`. Is `2 != val`? Yes, `2 != 3`.
 - `nums[k] = nums[i] => nums[1] = nums[2] => nums[1] = 2`.
 - Increment `k => k = 2`.
 - `nums` is now `[2, 2, 2, 3]`.
5. **i = 3:** `nums[3] = 3`. Is `3 != val`? No, `3 == 3`. Skip.
6. Loop ends.
7. Return `k` (which is 2). The array `nums` now has `[2, 2, _, _]` (the `_` indicate elements whose values don't matter beyond index `k-1`).

Java Code Solution

```
class Solution {
```

```

public int removeElement(int[] nums, int val) {
    // 'k' acts as the write pointer. It tracks the position where the next
    // non-'val' element should be placed. It also counts the number of
    // elements that are not equal to 'val'.
    int k = 0;

    // 'i' acts as the read pointer. It iterates through all elements
    // of the array.
    for (int i = 0; i < nums.length; i++) {
        // If the element at the current read pointer 'i' is NOT the value to remove,
        // we want to keep it.
        if (nums[i] != val) {
            // Place this non-'val' element at the 'k' position.
            // This effectively moves valid elements to the front of the array.
            nums[k] = nums[i];
            // Increment 'k' to point to the next available slot for a valid element.
            k++;
        }
        // If nums[i] == val, we simply skip it. It's effectively "removed"
        // because it won't be copied to the 'k' position.
        // The 'i' pointer will always advance to check the next element.
    }

    // After iterating through the entire array, 'k' holds the count
    // of elements that were not equal to 'val'. These elements are
    // now in the first 'k' positions of the 'nums' array.
    return k;
}
}

```

Python Code Solution

```

class Solution:
    def removeElement(self, nums: list[int], val: int) -> int:
        # 'k' acts as the write pointer. It tracks the position where the next
        # non-'val' element should be placed. It also counts the number of
        # elements that are not equal to 'val'.
        k = 0

        # 'i' acts as the read pointer. It iterates through all elements
        # of the list.
        for i in range(len(nums)):

```

```

# If the element at the current read pointer 'i' is NOT the value to remove,
# we want to keep it.
if nums[i] != val:
    # Place this non-'val' element at the 'k' position.
    # This effectively moves valid elements to the front of the list.
    nums[k] = nums[i]
    # Increment 'k' to point to the next available slot for a valid element.
    k += 1
# If nums[i] == val, we simply skip it. It's effectively "removed"
# because it won't be copied to the 'k' position.
# The 'i' pointer will always advance to check the next element.

# After iterating through the entire list, 'k' holds the count
# of elements that were not equal to 'val'. These elements are
# now in the first 'k' positions of the 'nums' list.
return k

```

Time Complexity (Optimal)

- **$O(N)$** : Both **i** and **k** pointers traverse the array/list at most **N** times. Each element is visited (read) once by the **i** pointer. Each non-**val** element is written once by the **k** pointer. This results in a linear time complexity proportional to the number of elements in **nums**.

Space Complexity (Optimal)

- **$O(1)$** : The solution modifies the input array/list in-place and uses only a few extra variables (**k**, **i**) regardless of the input size. This makes it an in-place solution with constant extra space.